



**POLYTECHNIQUE
MONTREAL**

UNIVERSITÉ
D'INGÉNIERIE

INF3500 - Conception et réalisation de systèmes numériques

Laboratoire 4 Pipelining

Florian DELHON

Hiver 2026

Introduction

À la fin de ce laboratoire, vous devrez être capable de :

- Concevoir et modéliser en VHDL un chemin de données qui réalise des fonctions arithmétiques et logiques, en intégrant l'instruction VHDL `for-generate` afin d'automatiser la génération des différents étages d'une architecture en pipeline
- Concevoir un banc d'essai VHDL pour valider ce chemin de données
- Effectuer la validation matérielle du chemin de données sur FPGA, après synthèse, implémentation et programmation

Ce laboratoire s'appuie sur et complémente le matériel suivant :

1. Concepts couverts dans les **Laboratoires 0, 1, 2 et 3**
2. Matière du cours « **Thème 11 - Latence, débit et surface** »

i Ce Thème 11 n'est présenté que début avril ?!

Ce n'est pas grave. Pour réaliser ce laboratoire, vous avez seulement besoin de savoir ce qu'est un pipeline et une boucle de génération en VHDL. Les aspects théoriques abordés en cours ne sont pas nécessaires ici. Seules quelques diapositives du Thème 11 vous seront utiles :

- les pp. 23–24 (Exercice) permettent de visualiser ce qu'est un pipeline et en quoi il permet une augmentation du débit,
- les pp. 44–45 (Instruction VHDL `for-generate`) décrivent la boucle de génération, avec un exemple concret qui vous sera très utile.

Liste des fichiers

Fichier	Contenu
src/division_par_reciproque.vhd	Module de division par la réciproque, déjà utilisé au Laboratoire 3
src/racine_carree_pipeline.vhd	Module de calcul pipeliné de la racine carrée
src/racine_carree_pipeline_tb.vhd	Banc d'essai du pipeline de calcul de la racine carrée
src/top.vhd	<i>Top-level</i> du projet
xdc/PYNQ-Z2-v1.0.xdc	Fichier de contraintes pour la PYNQ-Z2
synth-impl/create_zynq_wrapper.tcl synth-impl/zynq.tcl	Scripts <i>wrappant</i> le <i>top level</i> dans un design complet pour la plateforme PYNQ

Séance de laboratoire

Une séance de 3 heures sera dédiée à la réalisation de ce laboratoire : le **19 mars 2026 pour le groupe 2**, et le **26 mars 2026 pour le groupe 1**. Elle aura lieu à **8h30**, dans la salle **L-3714**.

Remise de code

Une remise de votre code est demandée sur votre dépôt **GitHub**. Pour ce faire, un *push* final contenant l'ensemble des fichiers nécessaires doit être fait avant le **1er avril 2026 à 22h** (ce n'est pas un poisson d'avril !), **pour les deux groupes**. Vous êtes libre de faire les *push* intermédiaires souhaités avant cette date. Il vous est demandé de **ne pas modifier les noms des fichiers/entités/architectures, ni la liste des generic/port**.

Entrevue/démonstration

Votre compréhension des travaux effectués dans ce laboratoire sera évaluée lors d'**une entrevue en présentiel**. Elle se tiendra au cours de la séance dédiée au laboratoire 5 : le **2 avril 2026 pour le groupe 2**, et le **16 avril 2026¹ pour le groupe 1**. Bien que l'entrevue se déroule en binôme, il sera demandé que **les deux élèves répondent de manière équilibrée** aux questions posées. Un élève n'ayant pas démontré une compréhension suffisante lors de l'entrevue pourra être pénalisé sur sa note individuelle.

¹Date résultant de l'ajustement du calendrier universitaire communiqué par Polytechnique le 17 mars 2026. Elle peut encore être sujette à changement.

Barème

Ce laboratoire sera noté sur 20, et comptera pour 4% de votre note finale.

Critères	Points
Remise de code	12
Partie 1 • Votre module <code>racine_carree_pipeline.vhd</code> fonctionnel	6
Partie 2 • Votre banc de test <code>racine_carree_pipeline_tb.vhd</code> complet	3
Partie 3 • Vos fichiers <code>labo4.ipynb</code>, <code>top.bit</code> et <code>top.hwh</code> fonctionnels	3
Entrevue/démonstration	8
Partie 1 • Votre conception de l'architecture <code>racine_carree_pipeline.vhd</code>	4
Partie 2 • Votre validation comportementale <code>racine_carree_pipeline_tb.vhd</code>	2
Partie 3 • Votre validation matérielle et démonstration sur la PYNQ-Z2	2
Total	20

! Évaluation

Pour vous aider à identifier les points évalués, vous pourrez vous fier aux encadrés rouges comme celui-ci. Veillez à bien les lire afin d'obtenir la note maximale.

Partie 0 : Préparation de l'environnement de travail

Initialisation du projet Vivado

Grâce aux sources fournies dans votre dépôt de départ, **initialisez un nouveau projet Vivado**. La procédure est détaillée dans la **Note 1**.

Partie 1 : Conception du pipeline de calcul

Contexte

Ce travail est une **suite du Laboratoire 3**. Vous y aviez implémenté une **architecture itérative** pour le calcul de la racine carrée (voir Figure 1). Cette approche directe vous a permis d'obtenir de bons résultats, avec une utilisation réduite des ressources. Dans ce laboratoire, vous allez mettre en place un **pipelining de ce calcul**.

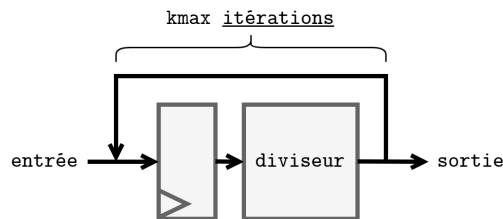


Fig. 1. – Schéma de l'architecture itérative du laboratoire 3

Pipelining

Le pipelining est un concept fondamental, très utilisé en architecture des ordinateurs. Il consiste à **découper un circuit en étages séparés par des registres**. Comme vous le verrez en cours, grâce au pipelining :

- le chemin critique combinatoire est brisé, ce qui permet une **augmentation de la fréquence d'opération maximale**.
- chaque étage accepte une nouvelle instruction à chaque cycle, ce qui permet une **augmentation du débit**.

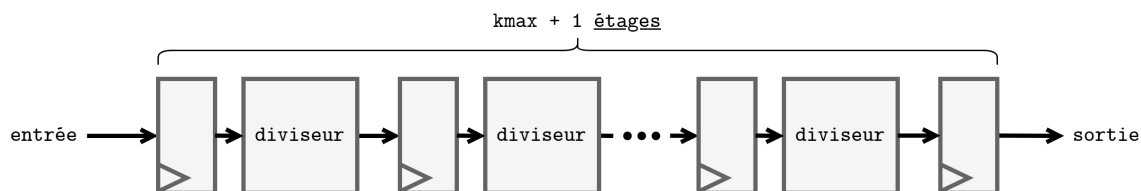


Fig. 2. – Schéma de l'architecture pipelinée à implémenter

À partir du module de racine carrée du **Laboratoire 3**, vous allez donc **dérouler la boucle dans l'espace** pour construire un pipeline à $k_{\max} + 1$ étages (voir Figure 2). Pour qu'un chemin de données pipeliné fonctionne, gardez à l'esprit qu'il faut que **les données entrées en même temps dans le pipeline en ressortent en même temps**.

Boucle de génération (for-generate) et tableaux (array)

Pour construire votre pipeline, **vous devrez utiliser une boucle de génération for-generate**. Elle permet d'automatiser la **génération de structures matérielles répétitives**, telles que celles rencontrées dans un pipeline. Contrairement au for-loop vu jusqu'ici, le **for-generate** permet l'instanciation paramétrique de composants. Vous en tirerez profit pour **instancier un diviseur_par_reciproque par étage**. Outre la boucle de génération, **utilisez des array** pour avoir une gestion claire de vos signaux, et n'hésitez pas à créer les types nécessaires.

AXI-Stream

Pour tirer profit de l'augmentation du débit évoquée plus tôt et des caractéristiques de la plateforme PYNQ, le **protocole de communication AXI-Stream** est la candidat parfait. Il permet le **transfert à haute vitesse de données en continu (flux de données)** entre un émetteur et un récepteur. Comme le montre l'exemple de la Figure 3, AXI-Stream repose principalement sur quatre signaux :

- **data** est la **donnée transférée**,
- **valid** indique qu'**une donnée est sur le bus**,
- **last** indique que **la dernière donnée du flux est sur le bus**,
- **ready** indique que **le receveur est prêt à recevoir** les données.

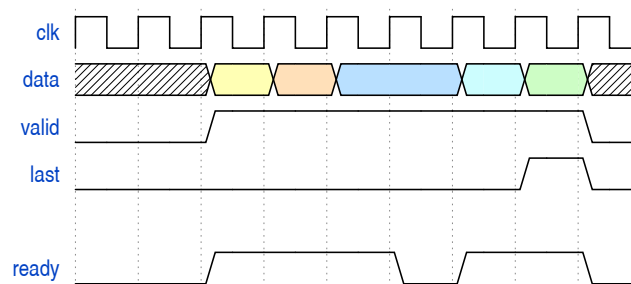


Fig. 3. – Exemple d'une séquence AXI-Stream

Concrètement, durant la conception de votre module, vous aurez à vous assurer que **le pipeline sera en pause lorsque le receveur n'est pas prêt (ready)**, et que les signaux **valid** et **last** seront bien transmis jusqu'à la sortie.

Et maintenant...

Implémentez votre pipeline de calcul de la racine carrée dans le fichier `racine_carree_pipeline.vhd`. Conservez les paramètres fournis dans le code de départ : $N = 16$, $M = 8$, $k_{\max} = 10$, $W_{\text{frac}} = 14$.

! Évaluation

- **Pour la remise de code**, *pushez* votre fichier `racine_carree_pipeline.vhd` fonctionnel sur votre dépôt GitHub.
- **Pour l’entrevue/démonstration**, soyez capable de présenter l’architecture de votre pipeline.

Partie 2 : Conception du banc d’essai

Un banc d’essai sommaire est fourni dans le fichier `racine_carree_pipeline_tb.vhd` : il teste les valeurs de 0 à 10, sans aucune assertion. Dans un premier temps, exécutez-le tel quel pour vérifier que votre pipeline fonctionne. Ensuite, **modifiez-le** pour qu’il applique le **même test exhaustif qu’au Laboratoire 3, avec les mêmes calculs d’erreur**. Notez que vous devriez aboutir aux **mêmes valeurs d’erreur qu’au Laboratoire 3**. **Attention** : pour votre calcul d’erreur, utilisez bien le signal `A_out` (relié à la sortie `o_A`) pour le calcul de la valeur théorique.

! Évaluation

- **Pour la remise de code**, *pushez* votre fichier `racine_carree_pipeline_tb.vhd` complet sur votre dépôt GitHub.
- **Pour l’entrevue/démonstration**, soyez capable de décrire votre logique de validation et de justifier le bon fonctionnement de votre module grâce aux résultats de simulation.

Partie 3 : Synthèse et implémentation sur la carte

Vous allez enfin effectuer la validation matérielle de votre module sur la carte. Vu qu’on souhaite ici utiliser le protocole AXI-Stream pour interagir avec le module, la procédure de validation dévie de qui a été vu dans les laboratoires précédents. **Suivez donc bien les étapes décrites ci-dessous**.

1. Dans Vivado, générez le *bitstream* comme habituellement.
2. Démarrez votre PYNQ-Z2, laissez-la se booter, et connectez-vous à l’interface Jupyter.
3. Sur l’interface Jupyter, créez un nouveau dossier.
4. Dans ce dossier, importez avec le bouton **Upload** les fichiers suivants :
 - le *bitstream*, généré dans `<project_dir>/<project_name>.runs/impl_1/`,
 - le *hardware handoff* (fichier `.hwh`), généré dans `<project_dir>/<project_name>.gen/sources_1/bd/zynq/hw_handoff/`, et que **vous renommerez `top.hwh`**.
5. Toujours dans ce dossier, créez un *notebook* `labo4.ipynb`, et ajoutez le code présenté à la page suivante.

```

# Import des librairies
import numpy as np
from pynq import allocate, Overlay

# Programmer le bitstream
ol = Overlay("top.bit")

# Allocation des buffers
ds = 4000
ibuf = allocate(shape=(ds,), dtype=np.uint32)
obuf = allocate(shape=(ds,), dtype=np.uint32)
for i in range(ds):
    ibuf[i] = i + 1
    # Vérification des valeurs
    if i < 10:
        print(f"{ibuf[i]},", end=' ')
print("\n---")

# Transfert des données vers le module AXI-Stream
ol.axi_dma_0.sendchannel.transfer(ibuf)
# Réception des résultats
ol.axi_dma_0.recvchannel.transfer(obuf)

# Afficher le contenu du buffer
for i in range(10):
    print(f"{obuf[i]},", end=' ')

```

Code 1. – Code permettant l'interaction par AXI-Stream avec votre module

Vous devriez voir les résultats s'afficher en sortie. Si c'est le cas, **modifiez ensuite ce code pour qu'il teste des valeurs représentatives que vous aurez judicieusement choisies.**

! Évaluation

- **Pour la remise de code**, *push*ez les trois fichiers qui vous ont servi à la validation matérielle : le `labo4.ipynb`, le `top.bit` et le `top.hwh`.
- **Pour l'entrevue/démonstration**, soyez capable de décrire en quelques mots votre démarche de validation dans le *notebook*, avec en point d'orgue la démonstration que votre calcul de racine carrée fonctionne sur la carte !